

Minime CI Workflow Architecture (Vertical Pipeline)

Unified GitHub Actions Build System — .github/workflows/build.yml [\[Shared\]](#)

Legend: [\[Shared\]](#) = Dual-distro / shared across targets | [\[Specific\]](#) = Single-distro, board, or libc specific

1. Trigger / Entrypoint

- **Workflow File:** .github/workflows/build.yml [\[Shared\]](#)
- **Triggers:** Push to main, Pull Request, or workflow_dispatch

↓ (Triggers parallel Stage 1 jobs)

2. Stage 1: Parallel Tier 1 Builds (Bootloader & UIs)

2A. Bootloader Build	2B. UI Build (musl)	2C. UI Build (glibc)
build-bootloader (AMD64) • Workflow Job: build-bootloader [Shared] • Runner: ubuntu-latest • Container: minime-glibc:latest [Shared] • Script: scripts/build-bootloader.sh [Shared] • Cache Key: bootloader-* (hash of uboot/**) • Configs & Patches: - minime/uboot/config/uboot.config [Shared] - minime/uboot/config/ddr3.defconfig [Specific] (H700) - minime/uboot/out/rk3566/rkbin/ [Specific] (RK3566) • GH Artifact: bootloader-out	build-ui (musl) (ARM64) • Workflow Job: build-ui (musl) [Shared] • Runner: ubuntu-24.04-arm • Container: minime-musl:latest [Shared] • Script: scripts/build-ui.sh minui allium musl [Shared] • Cache Key: ui-musl-* (hash of ui/**) • Submodules: - minime/ui/minui/ [Shared] (dual musl/glibc) - minime/ui/allium/ [Shared] (dual musl/glibc) • GH Artifact: ui-musl ■ minui-musl-aarch64.tar.xz ■ allium-musl-aarch64.tar.xz	build-ui (glibc) (AMD64) • Workflow Job: build-ui (glibc) [Shared] • Runner: ubuntu-latest • Container: minime-glibc:latest [Shared] • Script: scripts/build-ui.sh minui allium glibc [Shared] • Cache Key: ui-glibc-* (hash of ui/**) • Submodules: - minime/ui/minui/ [Shared] (dual musl/glibc) - minime/ui/allium/ [Shared] (dual musl/glibc) • GH Artifact: ui-glibc ■ minui-glibc-aarch64.tar.xz ■ allium-glibc-aarch64.tar.xz

↓ (Uploads artifacts to GH & Passes to Stage 2)

3. Artifact Transfer & Caching Layer

- **Actions Used:** actions/download-artifact@v4 [\[Shared\]](#), actions/cache/restore@v4 [\[Shared\]](#)
- **Artifacts Downloaded:** bootloader-out + matching UI artifact (ui-musl for Alpine, ui-glibc for Buildroot)
- **Tier 1 Output Caches:** Bootloader Cache (bootloader-*), UI Cache (ui-musl-*, ui-glibc-*)
- **Tier 2 OS Caches:** CCache (~/.alpine-ccache, ~/.buildroot-ccache) + DL Cache (~/.alpine-dl, ~/.buildroot-dl) [\[Specific\]](#)

↓ (Fans out to 6 parallel matrix jobs)

4. Stage 2: Target Component Compilation (build.sh)

4A. Alpine Component Builder (musl)	4B. Buildroot Component Builder (glibc)
Alpine (musl) Component Builder (3 Jobs) • Command: make -C minime/targets/alpine components BOARD=<board> [Specific] • Builder Script: minime/targets/alpine/scripts/build.sh [Specific] • Inputs: minime/boards/<board>/ [Specific], minime/boards/common/ [Shared] • Produced Artifacts: ■ system.erofs (EROFS rootfs: kernel, modules, GPU drivers)	Buildroot (glibc) Component Builder (3 Jobs) • Command: make -C minime/targets/buildroot components BOARD=<board> [Specific] • Builder Script: minime/targets/buildroot/scripts/build.sh [Specific] • Inputs: minime/boards/<board>/ [Specific], minime/boards/common/ [Shared] • Produced Artifacts: ■ system.erofs (EROFS rootfs: kernel, modules, GPU drivers)

↓ (Passes system.erofs to Packaging Stage)

5. Stage 2: Image Assembly & Packaging (mkimage.sh / mkupdate.sh)

5A. Alpine Image Assembly (musl)	5B. Buildroot Image Assembly (glibc)
Alpine Image & Update Packager (musl) • Commands: - make image BOARD=<board> UI=minui allium [Specific] - make update BOARD=<board> UI=minui allium [Specific] • Packaging Scripts: - minime/build/genassets.sh [Shared] - minime/build/mkimage.sh [Shared] - minime/build/mkupdate.sh [Shared] • Produced Release Archives: ■ *.img.xz (Raw SD card flashable image) ■ *.tar.xz (OTA update package)	Buildroot Image & Update Packager (glibc) • Commands: - make image BOARD=<board> UI=minui allium [Specific] - make update BOARD=<board> UI=minui allium [Specific] • Packaging Scripts: - minime/build/genassets.sh [Shared] - minime/build/mkimage.sh [Shared] - minime/build/mkupdate.sh [Shared] • Produced Release Archives: ■ *.img.xz (Raw SD card flashable image) ■ *.tar.xz (OTA update package)

↓ (Uploads output archives to GitHub Releases)

6. Release & Delivery Stage

- **Workflow Step:** Upload to Testing Release in .github/workflows/build.yml [\[Shared\]](#)
- **Tool / CLI:** gh release upload testing /*.img.xz /*.tar.xz --clobber [\[Shared\]](#)
- **Destination:** GitHub Releases — **testing** tag